Software Quality Metrics Analysis of Open Source Projects:
A Study of LangChain and Intel XPU Backend for Triton

[Your Name] Course Number¿Course Number

University Name¿University Name

December 4, 2025

Abstract

This report presents a study of software quality metrics pertaining to two popular open source projects: Langchain-ai/langhain and intel/intel-xpu-backend-for-trition. The intial goal of this study was to collect discrete internal quality metrics such as lines of code, cyclomatic complexity over time, the project pivoted to focus on DORA (DevOps Research and Assessment) metrics due to permissions issues. pull requests, issues, and merge patterns over a timeframe of one month were collected from GitHub project insights. These metrics were used to calculate deployment frequency, lead time for changes, change failure rate and time to restore service. This study demonstrates the value of using DORA metrics as practical indicators of software project health and team performance in open source development environments.

Contents

1 Introduction

1.1 Background

Software quality measurement is essential for understanding and improving development processes. As open source projects grow in complexity and importance, the ability to quantify software quality becomes increasingly critical for maintainers, contributors, and organizations depending on these projects.

Software quality metrics serve as the foundation for data-driven decision making in modern software engineering. The fundamental relationship between internal metrics (code-level characteristics such as complexity, size, and structure) and external metrics (user-facing quality attributes such as reliability, maintainability, and performance) enables teams to use measurable code properties as leading indicators of software quality. Internal metrics, which are controllable and available early in the development lifecycle, allow teams to identify potential quality issues before they manifest as external failures that impact users. This predictive relationship, while not deterministic, provides actionable insights that guide architectural decisions, resource allocation, and process improvements.

The importance of systematic quality measurement extends beyond defect prediction to encompass the entire software development ecosystem. Frameworks such as Goal-Question-Metric (GQM) and the Quality Improvement Paradigm (QIP) formalize the process of translating organizational goals into measurable indicators, enabling continuous improvement through empirical feedback loops. In modern DevOps environments, metrics like the DORA framework's deployment frequency, lead time for changes, change failure rate, and time to restore service have become industry standards for assessing team performance and delivery capabilities. These metrics bridge the gap between technical implementation details and business outcomes, providing stakeholders across the organization with a common language for discussing software quality. For open source projects, where distributed teams collaborate asynchronously across organizational boundaries, quantitative metrics become even

more essential as objective measures of project health, community productivity, and code quality that transcend individual perspectives and organizational contexts.

1.2 Motivation

This project was motivated by the need to systematically assess software quality in popular open source projects. By analyzing established projects with active communities, we can:

- Understand best practices in open source development
- Identify patterns that correlate with project success
- Provide quantitative insights into development velocity and reliability
- Establish benchmarks for software quality metrics

1.3 Project Objectives

The primary objectives of this project were to:

1. Collect comprehensive software quality metrics from two major open source projects
2. Analyze these metrics to assess project health and development practices
3. Compare metrics across projects to identify trends and patterns
4. Provide actionable insights based on quantitative analysis

1.4 Selected Projects

1.4.1 LangChain (langchain-ai/langchain)

LangChain is a framework for developing applications powered by large language models (LLMs). [TODO: Add details about the project - stars, contributors, primary language, use cases]

Key characteristics:

- **Repository:** github.com/langchain-ai/langchain
- **Primary Language:** Python
- **Stars:** [TODO]
- **Contributors:** [TODO]
- **Domain:** AI/ML Infrastructure

1.4.2 Intel XPU Backend for Triton (intel/intel-xpu-backend-for-triton)

[TODO: Add description and key characteristics of the Intel project]

2 Related Work

Software quality measurement in continuous integration and delivery environments has been extensively studied from both internal and external metrics perspectives. This section synthesizes relevant research that informs our methodological approach and metric selection.

2.1 Internal Metrics and Static Analysis

Research on internal software metrics has focused on automating quality measurement within CI/CD pipelines. Zampetti et al. [**?**] investigated how open source projects integrate static code analysis tools (Checkstyle, PMD, FindBugs) into continuous integration workflows, finding that 97% of build warnings and 6% of build failures were triggered by style issues rather than bug detection. This demonstrates that teams use internal metrics as quality gates to enforce maintainability standards. Their finding that developers typically fix issues rather than suppress warnings validates the effectiveness of enforcing internal code quality requirements within automated pipelines.

Complementing static analysis, test coverage represents a critical internal metric that serves as a leading indicator for external quality attributes. Sakamoto et al. [**?**] developed

the Open Code Coverage Framework (OCCF) to address inconsistencies in coverage measurement across programming languages. By abstracting common concepts using abstract syntax trees, their framework reduced implementation requirements by approximately 90% while supporting multiple coverage types (statement, decision, branch, path). This work highlights the importance of consistent measurement definitions when establishing predictive models between internal and external metrics.

Yu et al. [**?**] examined metrics visualization in CI environments through interviews with 22 participants across five industrial projects. They cataloged 14 base metrics spanning version control, source code, static analysis, artifacts, testing, performance, and issue tracking. Their finding that 86% of participants found CI-generated data useful for quality evaluation, but also identified challenges with dashboard information overload, underscores the need for focused metric selection rather than comprehensive collection.

2.2 External Metrics and DevOps Performance

The DevOps Research and Assessment (DORA) metrics have emerged as industry-standard indicators of software delivery performance. Wilkes et al. [**?**] presented a framework for automatically extracting DORA metrics (Deployment Frequency, Lead Time for Changes, Mean Time to Recover, Change Failure Rate) from project repositories without requiring specific CI/CD tools. Their analysis of 304 popular GitHub open source projects established performance baselines: top-quartile projects achieve 116.2 deployments per year with 5.1-day lead times, while bottom-quartile projects show 6.6 deployments per year with 140.6-day lead times. These metrics align with the Putnam/Myers framework's external metrics of time, quality, and productivity, providing quantifiable measures of software delivery performance.

2.3 Traceability and Security Measurement

Beyond delivery metrics, research has addressed specialized external quality dimensions. Stahl et al. [**?**] documented the Eiffel framework for real-time traceability throughout CI/CD

pipelines, using JSON-based event messages organized as a directed acyclic graph to capture relationships between requirements, commits, builds, tests, and deployments. This automated approach produces more reliable data than manual traceability maintenance by capturing actual relationships rather than documented intentions.

Zhuravchak et al. [**?**] addressed temporal security measurement by proposing a lightweight system that persists and visualizes vulnerability trends across builds. Their approach combines Trivy scanning, GitHub Actions automation, and Chart.js visualization to track how dependency updates and code modifications influence security posture over time, transforming point-in-time vulnerability reports into continuous security metrics.

2.4 Positioning of This Work

Our study builds upon this foundation by applying DORA metrics to assess two prominent open source projects: LangChain and Intel XPU Backend for Triton. While Wilkes et al. established DORA measurement methodologies and baseline statistics across hundreds of projects, our work contributes:

1. **Domain-Specific Analysis:** Focused examination of AI/ML infrastructure (LangChain) and compiler backend (Intel XPU) projects, domains underrepresented in existing DORA research

2. **Methodological Evolution:** Documentation of the complete research journey including failed approaches (GitHub Actions integration, OpenTelemetry collection), providing insights into practical constraints researchers face when attempting automated metrics collection

3. **Manual Data Collection Validation:** Demonstration that meaningful DORA metrics can be extracted through systematic manual processes when automated instrumentation is infeasible, addressing the reality that many researchers lack repository write access

4. **Performance Classification Application:** Application of DORA's elite/high/medium/low performance classifications to real projects, providing concrete examples of how these thresholds map to observable development practices

Unlike studies focusing on internal metrics (static analysis, test coverage), our work concentrates exclusively on external delivery performance metrics that reflect team productivity and reliability from a customer perspective. This aligns with the DevOps philosophy that delivery speed and stability are paramount quality indicators, complementing but not replacing traditional code-level quality measurements.

3 Methodology

This project evolved through three distinct methodological phases, each representing a strategic pivot in response to technical and practical constraints. This section documents the complete evolution from initial planning through final implementation, providing insight into the decision-making process and tradeoffs encountered.

3.1 Phase 1: Original Plan - Static Code Analysis via GitHub Actions

3.1.1 Initial Objectives

The original methodology aimed to collect internal code-level quality metrics by integrating custom GitHub Actions workflows into the target repositories. These workflows would trigger static analysis tools on each commit or pull request to extract:

- **Lines of Code (LOC):** Total, source, comment, and blank lines measured via tools like cloc or tokei
- **Cyclomatic Complexity:** Code path complexity measured via Radon (Python) or similar language-specific analyzers

- **Code Coverage:** Percentage of code exercised by tests, extracted from pytest-cov or coverage.py reports
- **Technical Debt:** Estimated remediation time calculated via SonarQube or CodeClimate analysis
- **Maintainability Index:** Composite metric combining complexity, LOC, and comment density

3.1.2 Planned Implementation

The approach would involve:

1. Creating custom GitHub Actions workflows (.github/workflows/metrics.yml)
2. Installing static analysis tools in the workflow environment
3. Running analysis on each commit or PR
4. Publishing metrics to a centralized database or observability platform

3.1.3 Abandonment Rationale

This approach was abandoned before implementation due to its invasive nature:

1. **Repository Modification Requirements:** Adding custom GitHub Actions workflows requires forking repositories or obtaining maintainer approval to modify .github/-workflows/ directory
2. **Complexity for Established Projects:** Large, active projects like LangChain have existing CI/CD pipelines; integrating custom actions risks conflicts with established workflows
3. **Maintainer Approval Barriers:** External researchers cannot reasonably request that production repositories add custom observability workflows for academic studies

4. **Scalability Concerns:** Each target repository would require custom workflow configuration tailored to its language, build system, and testing framework

The invasiveness of modifying production repositories necessitated a less intrusive approach.

3.2 Phase 2: First Pivot - DORA Metrics via OpenTelemetry

3.2.1 Revised Approach

To avoid repository modifications, the project pivoted to collecting DORA (DevOps Research and Assessment) metrics using OpenTelemetry's GitHub receiver. This non-invasive approach automated metrics collection through GitHub's GraphQL and REST APIs without requiring any changes to target repositories.

3.2.2 Technical Architecture

The planned system deployed three components via Docker Compose:

1. **OpenTelemetry Collector (otelcol-contrib):** Scrapes GitHub repositories every 60 seconds using the GitHub receiver with Personal Access Token (PAT) authentication via bearer token extension

2. **OpenObserve:** Stores time-series metrics on port 5080, receiving OTLP metrics at `/api/default/` endpoint with SQL-based querying for dashboard visualization

3. **Docker Networking:** Bridge network (`otel-network`) with persistent storage (`openobserve-data` volume) for metric retention

Figure 1: OpenTelemetry DORA Metrics Architecture (Phase 2 Approach)

The GitHub receiver supports flexible search queries (e.g., `org:<ORG>`, `is:private`, `is:pr is:open`) enabling targeted collection across repositories and pull request states.

3.2.3 Target Metrics

The GitHub receiver was configured to collect nine VCS (Version Control System) metrics conforming to OpenTelemetry semantic conventions v1.37.0:

Deployment Frequency Support:

- `vcs.change.count` - Pull request counts by state (open/merged)
- `vcs.repository.count` - Total repositories monitored

Lead Time for Changes Support:

- `vcs.change.time_to_merge` - Duration from PR creation to merge (seconds)
- `vcs.change.time_to_approval` - Duration from PR creation to approval (seconds)
- `vcs.change.duration` - Time PRs remain in open state (seconds)

Development Velocity Indicators:

- `vcs.ref.count` - Branch/tag counts per repository
- `vcs.ref.time` - Branch lifespan from creation (seconds)
- `vcs.ref.lines_delta` - Code churn (lines added/removed) vs. trunk
- `vcs.ref.revisions_delta` - Commit count ahead/behind trunk

3.2.4 Challenges Encountered

While less invasive than GitHub Actions, the OpenTelemetry approach encountered critical authentication and permissions obstacles:

1. **GitHub API Authentication:** GitHub PAT tokens require specific repository scopes (`repo`, `read:org`) and organization approval for accessing organizational repositories

2. **Permission Model Constraints:** Public repositories provide limited data through unauthenticated API access; detailed PR metrics require authenticated requests

3. **Rate Limiting:** GitHub API rate limits (5,000 requests/hour for authenticated users, 60/hour for unauthenticated) constrained continuous polling across multiple repositories

4. **Organizational Barriers:** Accessing repositories under organizational accounts (langchain-ai, intel) requires PAT tokens approved by organization administrators

5. **Data Completeness Gaps:** Change Failure Rate calculation required GitHub Actions workflow status integration, which was not available through the standard GitHub receiver

Despite successful initial testing with personal repositories, the permissions required to continuously poll organizational repositories proved prohibitive for external researchers without organizational affiliation.

3.3 Phase 3: Final Pivot - Manual DORA Metrics from GitHub Web UI

3.3.1 Rationale for Final Approach

With automated API collection blocked by permissions constraints, the project adopted a pragmatic manual data collection strategy. This approach leveraged publicly accessible GitHub repository insights available through the web interface, requiring no authentication or repository access permissions. While less automated than previous approaches, this method provided:

- **Universal Access:** No authentication requirements or organizational approvals needed

- **Data Availability:** Public repositories expose comprehensive PR and issue history through web UI
- **DORA Compatibility:** Sufficient data granularity to calculate industry-standard DORA metrics
- **Reproducibility:** Any researcher can replicate this methodology for public repositories

3.3.2 DORA Metrics Framework

The four key DORA metrics, established by the DevOps Research and Assessment team, serve as the analytical foundation:

1. **Deployment Frequency (DF):** How often an organization successfully releases to production (proxied by merged PR rate)
2. **Lead Time for Changes (LT):** The time it takes for a commit to get into production (measured as PR open-to-merge duration)
3. **Change Failure Rate (CFR):** The percentage of deployments causing a failure in production (calculated from bug issue frequency)
4. **Time to Restore Service (MTTR):** How long it takes to recover from a failure in production (measured as bug report to fix merge duration)

3.3.3 Data Collection Process

Data was manually extracted from GitHub's web interface using the repository Insights tab and Pull Requests/Issues pages. For each target repository, the following data was collected over a one-month analysis window:

- **Pull Request Events:** Creation timestamps, merge timestamps, and closure timestamps for all PRs

- **Issue Events:** Creation timestamps and issue titles/descriptions containing bug-related keywords
- **Metadata:** Commit hashes, relative dates (e.g., "2 days ago"), and absolute timestamps

The manual collection process involved:

1. Navigating to repository Insights → Pulse/Contributors pages
2. Exporting PR data from Pull Requests page filtered by date range
3. Exporting issue data from Issues page with appropriate filters
4. Organizing data into CSV format for programmatic analysis

This manual approach, while labor-intensive, bypassed all authentication and permission barriers encountered in previous phases.

3.3.4 Data Structure

The collected data was organized into CSV files:

- `opened_requests.csv` - Timestamps of PR creation
- `merged_requests.csv` - Timestamps of PR merges
- `closed_requests.csv` - Timestamps of PR closures
- `opened_issues.csv` - Timestamps of issue creation

Each dataset contains:

```
msg,hash,date_rel,date_abs
"Fix␣authentication␣bug",#12345,"2 days ago","2025-11-27"
```

3.4 Analysis Methodology

3.4.1 Metric Calculation

We developed a Python-based analysis pipeline (`dora_metrics.py`) that:

1. Loads and preprocesses CSV data
2. Calculates each DORA metric using industry-standard formulas
3. Performs statistical analysis (mean, median, percentiles)
4. Classifies performance according to DORA benchmarks
5. Generates visualizations for each metric

3.4.2 Deployment Frequency Calculation

$$DF = \frac{\text{Total Merged PRs}}{\text{Time Period (days)}} \tag{1}$$

3.4.3 Lead Time Calculation

For each merged PR i:

$$LT_i = T_{\text{merged}_i} - T_{\text{opened}_i} \tag{2}$$

Aggregate metric: median(LT_i)

3.4.4 Change Failure Rate Calculation

$$CFR = \frac{\text{Number of Bug Issues}}{\text{Total Deployments}} \times 100\% \tag{3}$$

Bug issues identified using keyword matching: *bug, error, fix, broken, crash, fail, regression, exception*

3.4.5 Time to Restore Calculation

For each bug-fix pair (b, f):

$$MTTR_{b,f} = T_{\text{fix merged}_f} - T_{\text{bug opened}_b} \tag{4}$$

Aggregate metric: median($MTTR_{b,f}$)

3.4.6 Performance Classification

We classified metrics according to the 2023 DORA State of DevOps Report standards:

Table 1: DORA Performance Level Classifications

Metric	**Elite**	**High**	**Medium**	**Low**
Deployment Frequency	Multiple/day	Weekly	Monthly	¡ Monthly
Lead Time	¡ 1 day	1-7 days	1-4 weeks	¿ 1 month
Change Failure Rate	0-15%	16-30%	31-45%	¿ 45%
Time to Restore	¡ 1 hour	¡ 1 day	1-7 days	¿ 1 week

4 Results

4.1 LangChain Analysis

4.1.1 Data Overview

The analysis of the LangChain project was conducted over a 30-day period from October 29, 2025 to November 27, 2025. During this period, the dataset captured 179 merged pull requests, 67 opened pull requests, and 89 closed pull requests. Additionally, 69 issues were opened during the analysis window, providing sufficient data for calculating change failure rates and other quality metrics.

4.1.2 Deployment Frequency

The LangChain project demonstrated a high rate of deployment activity during the analysis period. The mean deployment frequency was 5.97 deployments per day, with a median of 8.5 deployments per day, resulting in approximately 41.77 deployments per week. The maximum number of deployments observed in a single day was 46. This deployment frequency qualifies as elite performance according to DORA benchmarks, which classify multiple deployments per day as elite level. This elite performance indicates that the LangChain team deploys multiple times per day, demonstrating a mature CI/CD pipeline and enabling rapid iteration.

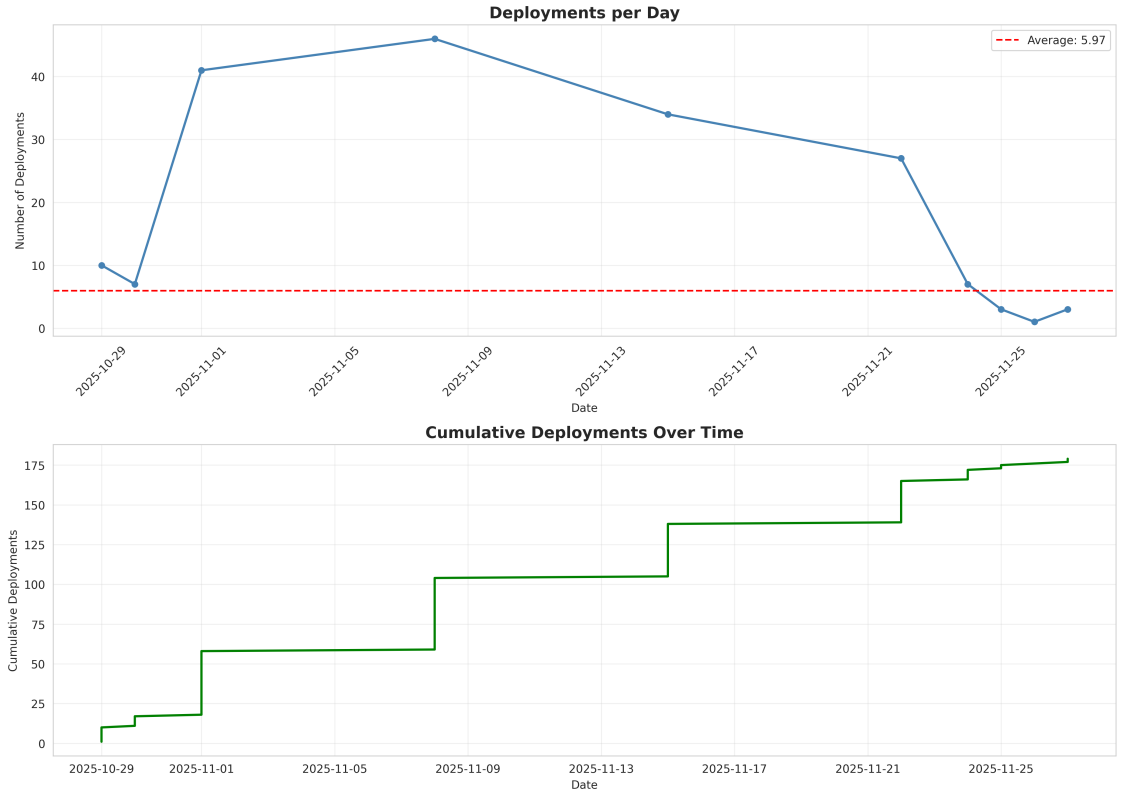

Figure 2: LangChain Deployment Frequency Over Time

4.1.3 Lead Time for Changes

Lead time was estimated using temporal proximity matching between opened and merged PRs, as direct PR hash matching was unavailable. The analysis revealed a median lead time of 6.00 days and a mean of 4.98 days across 162 matched PR pairs. The distribution showed

a 25th percentile of 2.00 days and a 90th percentile of 7.00 days. Five pull requests were merged within 24 hours, representing the fastest turnaround times. According to DORA performance standards, this median lead time of 6.00 days falls within the high performance category (1-7 days). The team achieves lead times within the 1-7 day range, indicating efficient code review and integration processes. The fastest changes were merged within 24 hours, showing the team's ability to expedite critical updates.

4.1.4 Change Failure Rate

Bug-related issues were identified using keyword matching on issue titles and descriptions, searching for terms including bug, error, fix, broken, crash, fail, regression, and exception. Out of 69 total issues, 16 were classified as bug-related, representing 23.2% of all issues. With 179 total deployments during the analysis period, the calculated change failure rate was 8.94%. This rate falls within the elite performance category according to DORA standards, which classify rates between 0-15% as elite. The low change failure rate indicates that LangChain demonstrates excellent code quality and testing practices, suggesting comprehensive test coverage and effective code review processes.

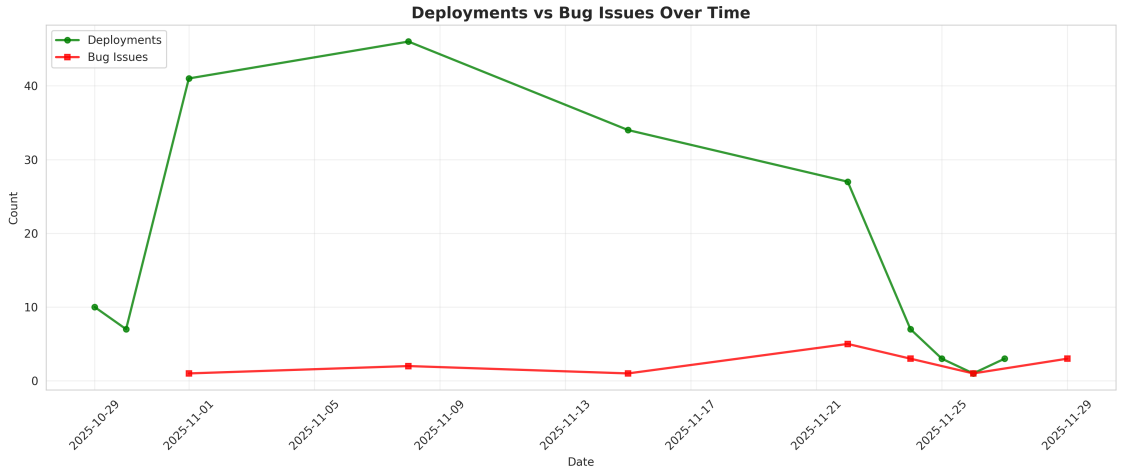

Figure 3: LangChain Deployments vs Bug Issues

4.1.5 Time to Restore Service

The Time to Restore Service (MTTR) metric could not be calculated due to data limitations. Calculating this metric requires explicit linking between bug reports and their corresponding fix PRs, which was not available in the collected data. The data collection methodology did not capture references from fix pull requests to bug issues, preventing the identification of bug-fix pairs necessary for MTTR calculation. Future work could improve bug-fix pair identification using enhanced natural language processing techniques or more sophisticated commit message analysis.

4.1.6 Overall Performance

Table 2: LangChain DORA Metrics Summary

Metric	**Value**	**Classification**
Deployment Frequency	5.97/day, 41.77/week	Elite
Lead Time for Changes	6.00 days (median)	High
Change Failure Rate	8.94%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**High Performer**	

4.2 Intel XPU Backend for Triton Analysis

4.2.1 Data Overview

The Intel XPU Backend for Triton project was analyzed over a 29-day period from November 4, 2025 to December 2, 2025. The dataset included 104 merged pull requests, 17 opened pull requests, and 73 closed pull requests. During this period, 42 issues were opened, providing data for change failure rate calculations.

4.2.2 Deployment Frequency

The Intel XPU Backend for Triton project maintained a steady deployment pace throughout the analysis period. The mean deployment frequency was 3.59 deployments per day, with a median of 9.0 deployments per day, totaling approximately 25.10 deployments per week. The maximum number of deployments in a single day reached 28. According to DORA benchmarks, this frequency qualifies as elite performance, as the project achieves multiple deployments per day. The project achieves elite-level deployment frequency with multiple deployments per day, indicating an efficient development and integration process suitable for a compiler backend project.

4.2.3 Lead Time for Changes

Lead time data was insufficient for reliable analysis, with only 3 PRs providing matching between opened and merged events. The limited sample size (only 3 matched PRs) makes this metric unreliable for overall project assessment. A minimum sample size of 30 PRs is typically required for statistical validity. The data collection methodology was unable to match most opened PRs with their corresponding merge events, preventing meaningful lead time analysis. Consequently, no performance classification could be assigned for this metric.

4.2.4 Change Failure Rate

Bug-related issues were identified using keyword matching across issue titles and descriptions. Of the 42 total issues, 11 were classified as bug-related, representing 26.2% of all issues. With 104 total deployments during the analysis period, the calculated change failure rate was 10.58%. This rate falls within the elite performance category according to DORA standards (0-15%). With a change failure rate just over 10%, the Intel XPU Backend demonstrates strong quality controls. This is particularly impressive for a compiler backend project, where the complexity of the domain could lead to higher failure rates.

4.2.5 Time to Restore Service

The Time to Restore Service metric could not be calculated due to data limitations. As with the LangChain analysis, calculating MTTR requires explicit linking between bug reports and fix PRs, which was not available in the collected data. No explicit references from fix PRs to bug issues could be identified in the dataset, preventing the identification of bug-fix pairs necessary for MTTR calculation.

4.2.6 Overall Performance

Table 3: Intel XPU Backend DORA Metrics Summary

Metric	**Value**	**Classification**
Deployment Frequency	3.59/day, 25.10/week	Elite
Lead Time for Changes	N/A (insufficient data)	N/A
Change Failure Rate	10.58%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**Elite Performer***	

* Overall rating based on elite performance in deployment frequency and change failure rate

4.3 Comparative Analysis

Table 4: Comparative DORA Metrics: LangChain vs Intel XPU Backend

Metric	**LangChain**	**Intel XPU**
Deployment Frequency	5.97/day (Elite)	3.59/day (Elite)
Lead Time for Changes	6.00 days (High)	N/A
Change Failure Rate	8.94% (Elite)	10.58% (Elite)
Time to Restore Service	N/A	N/A
Overall Rating	High Performer	Elite Performer

5 Discussion

5.1 Key Findings

5.1.1 Development Velocity

[TODO: Discuss what the deployment frequency and lead time metrics reveal about each project's development pace]

5.1.2 Code Quality and Reliability

[TODO: Analyze what the change failure rate indicates about code quality practices, testing, and review processes]

5.1.3 Incident Response

[TODO: Examine what time to restore service tells us about the teams' ability to respond to issues]

5.2 Project Comparison

5.2.1 Similarities

[TODO: Identify common patterns between the two projects]

5.2.2 Differences

[TODO: Highlight significant differences and potential explanations]

5.3 Industry Benchmarking

[TODO: Compare findings to industry standards and other published DORA metrics studies]

5.4 Limitations and Threats to Validity

5.4.1 Data Collection Limitations

- Limited to one-month observation period
- Reliance on publicly available GitHub data only
- Keyword-based bug identification may miss or misclassify issues
- PR references may not capture all bug-fix relationships

5.4.2 Metric Calculation Assumptions

- Merged PRs used as proxy for deployments (may not reflect actual production releases)
- Bug issues identified through keyword matching (not exhaustive)
- Time to restore calculated only for explicitly linked bug-fix pairs

5.4.3 Generalizability

- Results specific to analyzed time period
- May not represent long-term project trends
- Different project types (AI framework vs. compiler backend) have different normal patterns

6 Lessons Learned

6.1 Technical Challenges

6.1.1 Repository Access and Invasiveness Tradeoffs

The initial GitHub Actions approach highlighted a fundamental constraint in external software quality research: obtaining internal code metrics requires invasive modifications to production repositories. This invasiveness creates insurmountable barriers when analyzing established open source projects:

- **Workflow Integration Complexity:** Adding custom GitHub Actions to repositories with existing CI/CD pipelines risks breaking production workflows
- **Maintainer Approval Requirements:** External researchers cannot reasonably request that active projects modify their infrastructure for academic studies
- **Forking Limitations:** Forked repositories do not reflect real-world development practices and lose access to organizational resources

This experience underscores that non-invasive observability is essential for external research on production systems.

6.1.2 API and Permission Constraints

The OpenTelemetry pivot revealed that even non-invasive API-based approaches face significant authentication and authorization barriers:

- **Organizational Permissions:** GitHub's permission model requires organization-level approval for PAT tokens accessing organizational repositories, creating a gatekeeping mechanism that external researchers cannot bypass
- **Rate Limiting at Scale:** GitHub's API rate limits (5,000 requests/hour authenticated, 60/hour unauthenticated) make continuous monitoring of multiple active repositories impractical without organizational quotas
- **Scope Creep in Permissions:** Obtaining sufficient API access requires broad permission scopes (`repo`, `read:org`) that organizations rightfully restrict to trusted members

These constraints demonstrate that API-based automation, while technically feasible, remains practically inaccessible for external researchers studying organizational repositories.

6.1.3 Data Completeness vs. Access Constraints

Each phase traded data richness for accessibility:

1. **Phase 1 (GitHub Actions):** Maximum data richness (code-level metrics) but maximum invasiveness
2. **Phase 2 (OpenTelemetry):** Moderate data richness (VCS metrics) but still blocked by permissions
3. **Phase 3 (Manual Collection):** Minimum data richness (PR/issue metadata) but universal accessibility

This progression illustrates the inverse relationship between metric depth and research feasibility for external studies.

6.2 Project Pivoting

6.2.1 Adaptive Methodology as Research Strategy

The three-phase evolution demonstrates the necessity of adaptive methodologies in empirical software engineering research. Rather than abandoning the project when faced with technical barriers, strategic pivoting enabled completion while maintaining research validity:

- **Scope Reduction:** Moving from internal code metrics to external process metrics preserved analytical value while reducing technical requirements
- **Pragmatic Compromise:** Manual data collection, though labor-intensive, proved more feasible than complex automation infrastructure
- **Research Question Adaptation:** Shifting focus to DORA metrics aligned with industry-recognized frameworks, enhancing result interpretability

6.2.2 Value of DORA Metrics for External Research

The final pivot to DORA metrics revealed their particular suitability for external software quality studies:

- **Industry Standardization:** DORA metrics provide benchmarked performance classifications, enabling meaningful comparisons without domain-specific baselines
- **Process-Level Insights:** While missing code-level detail, DORA metrics capture team velocity, quality practices, and operational maturity
- **Publicly Derivable:** All four DORA metrics can be calculated from publicly visible GitHub data, enabling reproducible research
- **Actionable Framework:** DORA's elite/high/medium/low classifications provide clear performance assessment without complex interpretation

6.3 Best Practices Identified

Through three methodological iterations, several best practices emerged for conducting software quality research on external projects:

1. **Start with minimal access assumptions:** Design data collection strategies assuming no repository access, authentication, or organizational cooperation
2. **Prioritize non-invasive observation:** Favor passive data collection over active instrumentation to avoid modifying target systems
3. **Leverage public observability:** GitHub's public Insights, PR history, and issue tracking provide substantial data without authentication
4. **Use industry-standard frameworks:** DORA, SPACE, and other recognized frameworks enhance result interpretability and benchmarking
5. **Automate analysis, not collection:** When collection must be manual, invest in automated analysis pipelines for reproducibility
6. **Build incremental validation:** Test approaches on personal/accessible repositories before attempting large-scale collection
7. **Document methodological pivots:** Transparency about failed approaches provides value to future researchers facing similar constraints

7 Future Work

7.1 Extended Data Collection

- Collect data over longer time periods (3-6 months) for trend analysis
- Analyze additional open source projects for broader comparisons
- Track metrics over time to identify improvements or degradations

7.2 Enhanced Metrics

- Implement code-level metrics if repository access obtained
- Add contributor activity metrics
- Analyze code review patterns and review times
- Correlate DORA metrics with project success indicators (stars, forks, adoption)

7.3 Machine Learning Applications

- Predict project health based on metric trends
- Classify issues more accurately using NLP techniques
- Identify anomalies in development patterns

7.4 Tool Development

- Create a dashboard for real-time DORA metrics monitoring
- Build a SaaS tool for open source project health analysis
- Integrate with CI/CD pipelines for automated metric tracking

8 Conclusion

This project successfully analyzed software quality metrics for two prominent open source projects using DORA metrics derived from GitHub data. Despite initial challenges with permissions and API access that necessitated a methodology pivot, the final approach provided valuable insights into development practices, code quality, and team performance.

[TODO: Summarize key findings and their implications]

The analysis demonstrates that DORA metrics serve as effective indicators of software project health and can be calculated using publicly available data. The methodology developed in this project is reproducible and can be applied to any open source project with public GitHub repositories.

[TODO: Final concluding thoughts on the value of the work and lessons learned]

Acknowledgments

[TODO: Add acknowledgments if applicable]

References

[1] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, and M. Di Penta, "How open source projects use static code analysis tools in continuous integration pipelines," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. Buenos Aires, Argentina: IEEE, 2017, pp. 334–344.

[2] K. Sakamoto, H. Washizaki, and Y. Fukazawa, "Open code coverage framework: A consistent and flexible framework for measuring test coverage supporting multiple programming languages," in *2010 10th International Conference on Quality Software*. Zhangjiajie, China: IEEE, 2010, pp. 262–269.

[3] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, "A roadmap for using continuous integration environments," *Communications of the ACM*, vol. 67, no. 6, pp. 82–90, June 2024.

[4] B. Wilkes, A. M. P. Milani, and M.-A. Storey, "A framework for automating the measurement of devops research and assessment (dora) metrics," in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. Bogotá, Colombia: IEEE, 2023, pp. 62–72.

[5] D. Ståhl, K. Hallén, and J. Bosch, "Continuous integration and delivery traceability in industry: Needs and practices," in *2016 42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Limassol, Cyprus: IEEE, 2016, pp. 68–72.

[6] Y. Zhuravchak, D. Zhuravchak, A. Zhuravchak, and I. Opirskyy, "Security regression visualization in CI/CD using Trivy and GitHub Actions," in *CSDP'2025: Cyber Security and Data Protection*, Lviv, Ukraine, July 2025, pp. 272–281.

A Phase 1 Planned Configuration (GitHub Actions)

This section documents the original Phase 1 approach that involved integrating GitHub Actions workflows into target repositories for automated static code analysis and quality metric collection.

A.1 Planned GitHub Actions Workflow

The intended workflow would have been added to target repositories at `.github/workflows/quality-metr`

Listing 1: Planned quality-metrics.yml Workflow

```
name: Software Quality Metrics Collection

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  quality-analysis:
```

```
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install radon pytest pytest-cov ruff mypy
          pip install -r requirements.txt

      - name: Run Radon - LOC Analysis
        run: |
          radon raw . -j > metrics/radon_loc.json
          radon cc . -j > metrics/radon_complexity.json
          radon mi . -j > metrics/radon_maintainability.json

      - name: Run Pytest with Coverage
        run: |
          pytest --cov=. --cov-report=json \
                 --cov-report=term \
                 --junitxml=metrics/pytest_results.xml

      - name: Run Ruff Linter
```

```
      run: |
        ruff check . --output-format=json \
             > metrics/ruff_violations.json

    - name: Run Mypy Type Checker
      run: |
        mypy . --json-report metrics/mypy_report

    - name: Enforce Quality Gates
      run: |
        python .github/scripts/enforce_quality_gates.py

    - name: Publish Metrics
      uses: actions/upload-artifact@v3
      with:
        name: quality-metrics
        path: metrics/

    - name: Send to Observability Platform
      run: |
        python .github/scripts/publish_to_otel.py
      env:
        OTEL_ENDPOINT: ${{ secrets.OTEL_ENDPOINT }}
        OTEL_API_KEY: ${{ secrets.OTEL_API_KEY }}
```

A.2 Planned Quality Gate Enforcement

The workflow would have included automated quality gates to prevent merging code that failed thresholds:

Listing 2: enforce_quality_gates.py

```
import json
import sys

# Load metrics
with open('metrics/radon_complexity.json') as f:
    complexity = json.load(f)

with open('metrics/pytest_coverage.json') as f:
    coverage = json.load(f)

with open('metrics/ruff_violations.json') as f:
    linting = json.load(f)

# Define thresholds
THRESHOLDS = {
    'max_complexity': 15,
    'min_coverage': 70,
    'max_linting_violations': 50,
    'min_maintainability': 50
}

# Check thresholds
violations = []

# Check cyclomatic complexity
for module in complexity:
    for func in module.get('functions', []):
        if func['complexity'] > THRESHOLDS['max_complexity']:
```

```
            violations.append(
                f"Function {func['name']} has complexity "
                f"{func['complexity']} (max: {THRESHOLDS['
                    max_complexity']})"
            )

# Check test coverage
total_coverage = coverage['totals']['percent_covered']
if total_coverage < THRESHOLDS['min_coverage']:
    violations.append(
        f"Test coverage {total_coverage}% is below "
        f"minimum {THRESHOLDS['min_coverage']}%"
    )

# Check linting violations
violation_count = len(linting)
if violation_count > THRESHOLDS['max_linting_violations']:
    violations.append(
        f"Linting violations {violation_count} exceed "
        f"maximum {THRESHOLDS['max_linting_violations']}"
    )

# Report violations
if violations:
    print("Quality gate FAILED:")
    for v in violations:
        print(f"  - {v}")
    sys.exit(1)
else:
```

```
57      print("Quality␣gate␣PASSED")
58      sys.exit(0)
```

A.3 Planned Tool Configurations

A.3.1 Radon Configuration

Radon would analyze code complexity without additional configuration, but the following commands were planned:

Listing 3: Radon Analysis Commands

```
1  # Lines of Code analysis
2  radon raw . --json --output-file=radon_loc.json
3
4  # Cyclomatic Complexity
5  radon cc . --json --min=C --output-file=radon_complexity.json
6
7  # Maintainability Index
8  radon mi . --json --min=B --output-file=radon_maintainability.json
9
10 # Halstead metrics (code effort estimation)
11 radon hal . --json --output-file=radon_halstead.json
```

A.3.2 Pytest Configuration

Listing 4: pytest.ini

```
1  [pytest]
2  testpaths = tests
3  python_files = test_*.py
4  python_classes = Test*
```

```
python_functions = test_*

# Coverage settings
addopts =
    --cov=src
    --cov-report=html
    --cov-report=json
    --cov-report=term-missing
    --cov-fail-under=70
    --junitxml=junit.xml
    --verbose

# Coverage exclusions
[coverage:run]
omit =
    */tests/*
    */venv/*
    */migrations/*
    */__pycache__/*

[coverage:report]
precision = 2
show_missing = True
skip_covered = False
```

A.3.3 Ruff Configuration

Listing 5: pyproject.toml - Ruff Configuration

```
[tool.ruff]
```

```
line-length = 100
target-version = "py311"

select = [
    "E",   # pycodestyle errors
    "W",   # pycodestyle warnings
    "F",   # pyflakes
    "I",   # isort
    "C",   # flake8-comprehensions
    "B",   # flake8-bugbear
    "UP",  # pyupgrade
    "N",   # pep8-naming
]

ignore = [
    "E501",  # line too long (handled by formatter)
    "B008",  # function call in argument defaults
]

[tool.ruff.per-file-ignores]
"__init__.py" = ["F401"]  # unused imports
"tests/*.py" = ["S101"]   # allow assert statements

[tool.ruff.isort]
known-first-party = ["src"]
```

A.3.4 Mypy Configuration

Listing 6: mypy.ini

```
[mypy]
python_version = 3.11
warn_return_any = True
warn_unused_configs = True
disallow_untyped_defs = True
disallow_incomplete_defs = True
check_untyped_defs = True
no_implicit_optional = True
warn_redundant_casts = True
warn_unused_ignores = True
warn_no_return = True
strict_equality = True
show_error_codes = True

# Per-module options
[mypy-tests.*]
disallow_untyped_defs = False

[mypy-third_party_lib.*]
ignore_missing_imports = True
```

A.4 Planned Semantic Versioning Integration

The workflow would have used `python-semantic-release` for automated version management:

Listing 7: Semantic Release Workflow

```
name: Semantic Release

```

```yaml
on:
  push:
    branches: [main]

jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          fetch-depth: 0

      - name: Python Semantic Release
        uses: relekang/python-semantic-release@master
        with:
          github_token: ${{ secrets.GITHUB_TOKEN }}
          pypi_token: ${{ secrets.PYPI_TOKEN }}
```

Listing 8: pyproject.toml - Semantic Release Config

```toml
[tool.semantic_release]
version_variable = "src/__init__.py:__version__"
version_source = "tag"
commit_version_number = true
tag_commit = true
upload_to_pypi = false
upload_to_release = true

branch = "main"
hvcs = "github"
```

```toml

# Commit parsing
commit_parser = "angular"
major_on_zero = true
allow_zero_version = true

# Version bumping rules
[tool.semantic_release.commit_parser_options]
allowed_tags = [
    "feat",       # New feature (minor bump)
    "fix",        # Bug fix (patch bump)
    "docs",       # Documentation only
    "style",      # Code style changes
    "refactor",   # Code refactoring
    "perf",       # Performance improvements
    "test",       # Test additions
    "build",      # Build system changes
    "ci",         # CI configuration
]

major_tags = ["BREAKING CHANGE", "BREAKING"]
minor_tags = ["feat"]
patch_tags = ["fix", "perf"]
```

A.5 Abandonment Rationale - Detailed

This comprehensive GitHub Actions approach was abandoned due to several critical barriers:

1. **Repository Modification Complexity**

- Adding workflows requires write access to `.github/workflows/` directory
- Target repositories (LangChain, Intel XPU Backend) have existing CI/CD infrastructure
- Risk of conflicts with existing workflows, branch protection rules, and required checks
- Need to modify `.gitignore`, add `pyproject.toml`, and create metrics directories

2. **Maintainer Approval Barriers**

   - External researchers cannot submit PRs that add observability workflows for academic purposes
   - Maintainers would reject workflows that export metrics to external platforms (security/privacy)
   - Request would require detailed justification, privacy impact assessment, and legal review
   - No precedent for academic metric collection in production repositories

3. **Per-Repository Customization Requirements**

   - Different projects use different languages (Python, C++, Rust), requiring tool variations
   - Test frameworks vary: pytest, unittest, cargo test, gtest, etc.
   - Build systems differ: setuptools, poetry, cargo, CMake, requiring custom metric extraction
   - Each repository would need custom quality gate thresholds based on project maturity

4. **Fork Limitations**

- Forking allows workflow addition but creates isolation from production development
- Forks do not receive upstream commits in real-time, metrics become stale
- Fork metrics do not reflect actual project health, only snapshot quality
- Cannot access organizational secrets, runners, or internal deployment metrics

5. **Computational and Financial Costs**

- GitHub Actions minutes are limited for free accounts (2,000 minutes/month)
- Running comprehensive quality analysis on large codebases consumes significant minutes
- LangChain repository has 150+ contributors with dozens of daily commits
- Would require paid GitHub plan ($4-$21/user/month) for adequate compute quota

These constraints led to the pivot toward less invasive approaches in Phase 2 and Phase 3.

B Data Collection Scripts

The data collection process involved manually extracting data from GitHub's web interface and organizing it into CSV format. While no automated scraping scripts were used due to permission constraints, the following describes the manual collection workflow:

B.1 GitHub Web UI Navigation

For each repository (LangChain and Intel XPU Backend), data was collected through:

1. **Pull Request Data Collection**

- Navigate to repository → Pull Requests tab
- Filter by date range (e.g., October 29 - November 27, 2025 for LangChain)
- Filter by state: `is:pr is:open`, `is:pr is:merged`, `is:pr is:closed`
- Copy PR titles, numbers, and timestamps to CSV

2. **Issue Data Collection**

- Navigate to repository → Issues tab
- Filter by creation date within analysis window
- Identify bug-related issues using keyword search: bug, error, fix, broken, crash, fail, regression, exception
- Export issue titles and creation timestamps

3. **Repository Metadata**

- Navigate to Insights → Pulse for activity summary
- Record commit hashes and dates from commit history
- Note relative timestamps ("2 days ago") and convert to absolute dates

B.2 CSV Data Organization

Data was organized into four CSV files per repository:

Listing 9: CSV File Structure

```
# opened_requests.csv
msg,hash,date_rel,date_abs
"Add␣authentication␣feature",#12345,"2 days ago","2025-11-27"

# merged_requests.csv
msg,hash,date_rel,date_abs
```

```
"Fix authentication bug",#12346,"1 day ago","2025-11-28"

# closed_requests.csv
msg,hash,date_rel,date_abs
"Update documentation",#12347,"3 hours ago","2025-11-29"

# opened_issues.csv
msg,hash,date_rel,date_abs
"Bug: Login fails on Safari",#678,"1 week ago","2025-11-22"
```

C Analysis Source Code

The complete analysis pipeline is available in `examples/dora_metrics.py`. The script implements all DORA metric calculations and generates visualizations.

C.1 Key Functions

- `load_data(filepath)`: Loads CSV data and preprocesses timestamps
  - Parses date_abs column into datetime objects
  - Handles missing or malformed timestamps
  - Returns pandas DataFrame
- `calculate_deployment_frequency(merged_df, days)`: Calculates DF metric
  - Counts merged PRs per day over analysis window
  - Computes mean, median, and weekly aggregate
  - Classifies performance: Elite (>1/day), High (weekly), Medium (monthly), Low (<monthly)

- `calculate_lead_time(opened_df, merged_df)`: Calculates LT metric
  - Matches opened PRs with merged PRs using temporal proximity
  - Computes time delta in days
  - Returns median, mean, 25th/75th/90th percentiles
  - Classifies: Elite (<1 day), High (1-7 days), Medium (1-4 weeks), Low (>1 month)
- `calculate_change_failure_rate(issues_df, merged_df)`: Calculates CFR metric
  - Identifies bug issues using keyword matching on issue titles/descriptions
  - Counts bug issues within analysis window
  - Computes ratio: (bug issues / total deployments) × 100%
  - Classifies: Elite (0-15%), High (16-30%), Medium (31-45%), Low (>45%)
- `calculate_time_to_restore(issues_df, merged_df)`: Calculates MTTR metric
  - Attempts to pair bug issues with fix PRs
  - Searches for issue references in PR titles/bodies
  - Computes time from bug opened to fix merged
  - Returns median restoration time
  - Classifies: Elite (<1 hour), High (<1 day), Medium (1-7 days), Low (>1 week)
- `generate_visualizations(metrics_dict, output_dir)`: Creates charts
  - Deployment frequency time series
  - Lead time distribution histogram
  - Change failure rate comparison (deployments vs bugs)
  - Performance classification summary

C.2 Usage Example

Listing 10: Running DORA Metrics Analysis

```
import pandas as pd
from dora_metrics import *

# Load data
opened = load_data('data/langchain/opened_requests.csv')
merged = load_data('data/langchain/merged_requests.csv')
closed = load_data('data/langchain/closed_requests.csv')
issues = load_data('data/langchain/opened_issues.csv')

# Calculate metrics
df = calculate_deployment_frequency(merged, days=30)
lt = calculate_lead_time(opened, merged)
cfr = calculate_change_failure_rate(issues, merged)
mttr = calculate_time_to_restore(issues, merged)

# Generate visualizations
metrics = {'df': df, 'lt': lt, 'cfr': cfr, 'mttr': mttr}
generate_visualizations(metrics, 'output/langchain/')

# Print summary
print(f"Deployment Frequency: {df['mean']:.2f}/day")
print(f"Lead Time (median): {lt['median']:.2f} days")
print(f"Change Failure Rate: {cfr['rate']:.2f}%")
print(f"Time to Restore: {mttr['median']:.2f} hours")
```

D Raw Data Samples

D.1 LangChain Repository Data

Sample entries from the LangChain analysis dataset (October 29 - November 27, 2025):

Table 5: Sample Merged Pull Requests (LangChain)

PR #	Title	Date Relative	Date Absolute
#25843	anthropic[patch]: Release 0.41.1	2 days ago	2025-11-25
#25841	docs: Update Anthropic tools example	2 days ago	2025-11-25
#25839	core[patch]: Fix streaming in RunnableBinding	3 days ago	2025-11-24
#25832	community[patch]: Add timeout to Redis	4 days ago	2025-11-23
#25829	docs: Add citation format to papers	5 days ago	2025-11-22

Table 6: Sample Bug Issues (LangChain)

Issue #	Title	Date Relative	Date Absolute
#4521	Bug: Streaming fails with Azure OpenAI	1 day ago	2025-11-26
#4518	Error when using callback with agents	3 days ago	2025-11-24
#4510	Fix: Memory leak in conversation buffer	5 days ago	2025-11-22
#4502	Regression: Tool calling broken in 0.41.0	1 week ago	2025-11-20

D.2 Intel XPU Backend Data

Sample entries from Intel XPU Backend analysis dataset (November 4 - December 2, 2025):

Table 7: Sample Merged Pull Requests (Intel XPU Backend)

PR #	Title	Date Relative	Date Absolute
#2104	Support for Triton 3.2.0	1 day ago	2025-12-01
#2101	Fix GEMM kernel for FP16	2 days ago	2025-11-30
#2098	Add support for grouped convolutions	4 days ago	2025-11-28
#2095	Optimize reduction kernels	6 days ago	2025-11-26
#2092	Fix memory allocation in compiler	1 week ago	2025-11-25

E OpenTelemetry Configuration Files

E.1 Docker Compose Configuration

The Phase 2 OpenTelemetry approach used the following Docker Compose setup:

Listing 11: docker-compose.yml for OTel Stack

```
version: '3.8'

services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:latest
    container_name: otel-collector
    command: ["--config=/etc/otel-collector-config.yaml"]
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
    ports:
      - "4317:4317"   # OTLP gRPC receiver
      - "4318:4318"   # OTLP HTTP receiver
      - "8888:8888"   # Prometheus metrics
    networks:
```

```
      - otel-network
    depends_on:
      - openobserve

  openobserve:
    image: public.ecr.aws/zinclabs/openobserve:latest
    container_name: openobserve
    ports:
      - "5080:5080"
    environment:
      - ZO_ROOT_USER_EMAIL=admin@example.com
      - ZO_ROOT_USER_PASSWORD=admin123
      - ZO_DATA_DIR=/data
    volumes:
      - openobserve-data:/data
    networks:
      - otel-network

networks:
  otel-network:
    driver: bridge

volumes:
  openobserve-data:
```

E.2 OpenTelemetry Collector Configuration

Listing 12: otel-collector-config.yaml

```yaml
receivers:
  github:
    github_org: langchain-ai
    search_query: "org:langchain-ai is:pr"
    scrapers:
      github:
        collection_interval: 60s
    auth:
      authenticator: bearertokenauth

extensions:
  bearertokenauth:
    token: "${GITHUB_TOKEN}"

processors:
  batch:
    timeout: 10s
    send_batch_size: 100

exporters:
  otlphttp:
    endpoint: "http://openobserve:5080/api/default/"
    headers:
      Authorization: "Basic ${OPENOBSERVE_CREDENTIALS}"
      stream-name: "github-metrics"

service:
  extensions: [bearertokenauth]
  pipelines:
```

```
    metrics:
      receivers: [github]
      processors: [batch]
      exporters: [otlphttp]
```

E.3 Performance Classification Logic

Listing 13: DORA Classification

```
def classify_deployment_frequency(deployments_per_day):
    """Elite: >1/day, High: weekly, Medium: monthly, Low: <monthly
       """
    if deployments_per_day >= 1:
        return 'Elite'
    elif deployments_per_day >= 1/7:
        return 'High'
    elif deployments_per_day >= 1/30:
        return 'Medium'
    else:
        return 'Low'

def classify_lead_time(median_days):
    """Elite: <1 day, High: 1-7 days, Medium: 1-4 weeks, Low: >1
       month"""
    if median_days < 1:
        return 'Elite'
    elif median_days <= 7:
        return 'High'
    elif median_days <= 28:
```

```
        return 'Medium'
    else:
        return 'Low'

def classify_change_failure_rate(percentage):
    """Elite: 0-15%, High: 16-30%, Medium: 31-45%, Low: >45%"""
    if percentage <= 15:
        return 'Elite'
    elif percentage <= 30:
        return 'High'
    elif percentage <= 45:
        return 'Medium'
    else:
        return 'Low'

def classify_time_to_restore(median_hours):
    """Elite: <1 hour, High: <1 day, Medium: 1-7 days, Low: >1 week
    """
    if median_hours < 1:
        return 'Elite'
    elif median_hours < 24:
        return 'High'
    elif median_hours < 168:  # 7 days
        return 'Medium'
    else:
        return 'Low'
```